

JEGYZŐKÖNYV

Modern adatbázis rendszerek MSc

XQuery és ORDBMS technológiák vizsgálata

Készítette: Deme Zalán

Neptunkód: NDZDH6

Dátum: 2026. május 10.

Tartalomjegyzék

1. Választott téma: XQuery	1
1.1. A feladat leírása	1
1.2. Elméleti háttér	1
1.2.1. XQuery	1
1.2.2. Az XDM adatmodell	1
1.2.3. A FLWOR kifejezések felépítése	2
1.2.4. XQuery az eXist-db környezetben	2
1.2.5. XQuery és a relációs lekérdezések összehasonlítása	2
1.3. A futtatás eredménye	3
1.3.1. Szimpla lekérdezés	3
1.3.2. Komplex lekérdezés	3
1.3.3. Adatok összekapcsolása (JOIN)	4
1.3.4. Adatmódosítás (DML)	5
1.3.5. Aggregációs lekérdezések	5
2. ORDBMS – Elméleti rész	6
2.1. Procedurális komponensek az Oracle UDT-kben	6
2.2. Öröklés, polimorfizmus	6
2.3. IS OF és TREAT operátorok	7
3. Gyakorlati feladatok megoldása	8
3.1. Alaptípus létrehozása (szemely_t)	8
3.2. Öröklés: foszakacs_t és vendeg_t típusok	8
3.3. Objektumtábla létrehozása és feltöltése XML-ből	9
3.4. Polimorf lekérdezések	9
3.5. szakacs2_t altípus integrálása	10

1. fejezet

Választott téma: XQuery

1.1. A feladat leírása

A félév önálló feladatának első részeként az XQuery lekérdezőnyelv témakörét választottam, ahol a következő téákat fejtem ki röviden:

- Alapvető információk az XQuery-ről
- Az XQuery szintaxis és a FLWOR kifejezések felépítése
- Az eXist-db natív XML adatbázis-kezelő használata, lekérdezések megvalósítása
- Gyakorlati példák készítése az saját XML dokumentumon (egyetem.xml), eXist-db felületen futtatva.

1.2. Elméleti háttér

1.2.1. XQuery

Az XQuery (XML Query Language) szerepe az XML dokumentumoknál ugyanaz, mint a relációs adatbázisoknál az SQL-é. Ezt a nyelvet a W3C azért hozta létre, hogy legyen egy egységes, szabványos eszköz az XML alapú adatok keresésére, kinyerésére és formázására.

Míg az SQL logikailag táblákból és sorokból áll, az XQuery-t kifejezetten a hierarchikus, fa szerkezetű adatok kezelésére optimalizálták. Legyen szó egy egyszerű konfigurációs fájlról vagy egy nagy méretű adatbázisról, az XQuery segítségével könnyen lehet navigálni az adatok között.

1.2.2. Az XDM adatmodell

Az XQuery nem egyszerű szöveges fájlként kezeli az XML dokumentumokat, hanem az úgynevezett XDM (XQuery and XPath Data Model) logikai adatmodell alapján értelmezi azokat.

Ebben a modellben a rendszer a dokumentumot egy logikai fává alakítja. Minden adat egy szekvenciát alkot, amely a következőkből állhat:

- **Csomópontok:** Ezek a fa építőelemei, például az XML elemek (tagek), az attribútumok vagy a szöveges tartalmak.

- **Atomi értékek:** Egyszerű adattípusok, mint például egy szám vagy egy karakter sorozat.

Ezzel a logikai fa-struktúrával az XQuery hatékonyan képes a komplex, mélyen beágyazott összefüggések felismerésére anélkül, hogy a teljes fájl karakterről karakterre elemeznie kellene.

1.2.3. A FLWOR kifejezések felépítése

Az utasítások felépítése FLWOR mozaikszóval van jelölve, ami a sorrendet mutatja meg. Ez a kifejezéslánc teszi lehetővé az adatok szűrését, módosítását és összekapcsolását:

- **FOR:** Iterál egy szekvencia elemein.
- **LET:** Változót köt egy teljes szekvenciához.
- **WHERE:** Logikai szűrőfeltételeket határoz meg.
- **ORDER BY:** Meghatározza a kimeneti sorrendet (növekvő vagy csökkenő) valami változó alapján.
- **RETURN:** Definiálja a kimeneti struktúrát, ahol új XML tagek is létrehozhatók.

Példa egy alapvető FLWOR lekérdezésre az egyetemi rendszeremben:

Programkód 1.1. Tárgyak szűrése kredit alapján

```
for $t in doc("/db/egyetem.xml")//targy
let $k := $t/kredit
where $k > 4
order by $t/nev
return
  <eredmeny>
    { $t/nev }
    <kredit_ertek>{ data($k) }</kredit_ertek>
  </eredmeny>
```

1.2.4. XQuery az eXist-db környezetben

Az eXist-db egy natív XML adatbázis-kezelő rendszer, amely az adatokat nem táblákba tördelve, hanem eredeti hierarchikus formájukban tárolja. Ellentétben a relációs adatbázisokkal (ahol az XQuery-t SQL-be kell ágyazni), az eXist-db-ben a lekérdezések közvetlenül futtathatóak az adatokat tartalmazó fájlokon. Ez jelentős teljesítménynövekedést biztosít.

1.2.5. XQuery és a relációs lekérdezések összehasonlítása

Bár az SQL és az XQuery célja hasonló (adatkinyerés), a megközelítésük eltér:

	Relációs (SQL)	XML (XQuery)
Adatmodell	Lapos táblák, rekordok	Hierarchikus fa struktúra
Kapcsolatok	Join műveletek	Kapcsolótábla, beágyazás
Séma	Merev (Schema-on-write)	Rugalmas (Schema-on-read)
Környezet	pl.: Oracle Apex, SQL Plus	eXist-db, XQuery Sandbox

1.3. A futtatás eredménye

A gyakorlati feladat során az egyetemi rendszer adatait tartalmazó XML fájlt töltöttem fel az eXist-db adatbázisba, majd az alábbi lekérdezéseket futtattam a rendszer beépített felületén.

1.3.1. Szimpla lekérdezés

A szimpla lekérdezés során a rendszerben tárolt összes hallgató adatait kértem le szűrés nélkül.

Programkód 1.2. Összes hallgató lekérdezése

```
doc("/db/egyetem.xml")//hallgato
```

```

1 <hallgato sid="s1">
  <nev>Kovács László</nev>
  <lakcim>
    <varos>Budapest</varos>
    <utca>Petőfi Sándor</utca>
    <hazszam>10</hazszam>
  </lakcim>
  <email>kovacs.laszlo@bme.hu</email>
  <szuletesi_ido>1995-05-20</szuletesi_ido>
</hallgato>
2 <hallgato sid="s2">
  <nev>Szűcs Eszter</nev>
  <lakcim>
    <varos>Debrecen</varos>
    <utca>Kossuth Lajos utca</utca>
    <hazszam>5</hazszam>
  </lakcim>
  <email>szucs.eszter@de.edu</email>
  <szuletesi_ido>1996-08-15</szuletesi_ido>
</hallgato>
3 <hallgato sid="s3">
  <nev>Horváth Péter</nev>
  <lakcim>
    <varos>Szeged</varos>
    <utca>Rákóczi Ferenc utca</utca>
    <hazszam>15</hazszam>
  </lakcim>
  <email>horvath.peter@szeged.edu</email>
  <szuletesi_ido>1997-03-10</szuletesi_ido>
</hallgato>

```

1.1. ábra: Egyszerű lekérdezés eredménye

1.3.2. Komplex lekérdezés

Komplex lekérdezőként olyan kurzusokat kerestem, amelyek legalább 20 fő férőhelyesek és magyar nyelvűek.

Programkód 1.3. Magyar nyelvű kurzusok szűrése

```
for $k in doc("/db/egyetem.xml")//kurzus
```

```

where $k/nyelv = 'magyar' and $k/max_fo >= 20
return
  <kurzus_adat>
    <Kod>{data($k/@kurzuskod)}</Kod>
    <Letszam>{data($k/max_fo)}</Letszam>
  </kurzus_adat>

```

```

1 <kurzus_adat>
  <Kod>k1</Kod>
  <Letszam>100</Letszam>
</kurzus_adat>
2 <kurzus_adat>
  <Kod>k3</Kod>
  <Letszam>20</Letszam>
</kurzus_adat>

```

1.2. ábra: Komplex lekérdezés eredménye

1.3.3. Adatok összekapcsolása (JOIN)

Ahogy az elméleti részben említettem, a hallgatók és beiratkozásaik összekapcsolását beágyazott ciklussal oldottam meg. A `let` segítségével gyűjtöttem ki az adott hallgatóhoz tartozó beiratkozásokat a közös azonosító (`sid`) alapján.

Programkód 1.4. Hallgatók és beiratkozások összekapcsolása

```

for $h in doc("/db/egyetem.xml")//hallgato
let $beiratkozások := doc("/db/egyetem.xml")//beiratkozás[@sid = $h/@sid]
for $b in $beiratkozások
return
  <hallgato_beiratkozás>
    <nev>{data($h/nev)}</nev>
    <kepzes>{data($b/kepzes_neve)}</kepzes>
    <statusz>{data($b/statusz)}</statusz>
  </hallgato_beiratkozás>

```

```

1 <hallgato_beiratkozás>
  <nev>Kovács László</nev>
  <kepzes>BSc Informatika</kepzes>
  <statusz>aktív</statusz>
</hallgato_beiratkozás>
2 <hallgato_beiratkozás>
  <nev>Kovács László</nev>
  <kepzes>BSc Biológia</kepzes>
  <statusz>aktív</statusz>
</hallgato_beiratkozás>
3 <hallgato_beiratkozás>
  <nev>Szűcs Eszter</nev>
  <kepzes>BSc Biológia</kepzes>
  <statusz>passzív</statusz>
</hallgato_beiratkozás>
4 <hallgato_beiratkozás>
  <nev>Horváth Péter</nev>
  <kepzes>BSc Fizika</kepzes>
  <statusz>aktív</statusz>
</hallgato_beiratkozás>

```

1.3. ábra: Kapcsolt lekérdezés eredménye

1.3.4. Adatmódosítás (DML)

Az adatok módosítását a megfelelő update szintaktikával végeztem el, itt a return ágban kell a frissítést elvégezni. Az alábbi példában minden tárgy kreditjét egységesen megnöveltem eggyel.

Programkód 1.5. Kreditértékek módosítása

```
for $t in doc("/db/egyetem.xml")//targy
return update value $t/kredit with ($t/kredit + 1)
```

Ilyen esetben látható eredmény nincs, az existDB felülete ad egy visszajelzést a művelet sikerességéről.

1.3.5. Aggregációs lekérdezések

Az XQuery beépített függvényei lehetővé teszik matematikai statisztikák készítését a dokumentumon. Ezekre példa a számláló és a max függvény:

Programkód 1.6. Aggregációs példák

```
(: osszes beiratkozas szama :)
count(doc("/db/egyetem.xml")//beiratkozas)

(: legnagyobb kreditszam :)
max(doc("/db/egyetem.xml")//targy/kredit)/number()
```

A visszaadott eredmény helyes, de rossz számrendszerben érkezik vissza, ez más rendszerben nem így történik.

2. fejezet

ORDBMS – Elméleti rész

2.1. Procedurális komponensek az Oracle UDT-kben

Az Oracle objektum-relációs adatbázis-rendszer (ORDBMS) lehetővé teszi, hogy a felhasználó által definiált típusok, azaz az UDT (User Defined Type) ne csak adatot, hanem üzleti logikát is tartalmazzon. Ezt a típus törzse (TYPE BODY) valósítja meg, amelyben a következő procedurális komponensek definiálhatók:

- **MEMBER FUNCTION / PROCEDURE:** A típus egy konkrét példányán hívható. A MEMBER FUNCTION értéket ad vissza, a MEMBER PROCEDURE nem. Mindkettő hozzáfér az aktuális példányhoz a SELF kulcsszón keresztül. A MEMBER FUNCTION meghívható SELECT és WHERE használatokkor is.
- **STATIC FUNCTION / PROCEDURE:** Példányosítás nélkül hívható, közvetlenül a típusnéven keresztül (pl. `szemely_t.min_munkakor_eletkor()`). Nincs SELF referencia, így nem fér hozzá egyetlen konkrét objektum adataihoz sem. Tipikus használata: konstansok, osztályszintű segédfüggvények.
- **MAP / ORDER MEMBER FUNCTION:** Az Oracle alapértelmezés szerint nem tudja összehasonlítani az objektumokat (pl. rendezéshez vagy egyenlőségvizsgálathoz). Ezért definiálni kell egy rendező függvényt. A MAP MEMBER FUNCTION az objektumot egy skaláris értékre képezi le; az ORDER MEMBER FUNCTION közvetlenül két objektumot hasonlít össze, és -1, 0 vagy +1 értéket ad vissza. Egyszerre csak az egyik megengedett.

2.2. Öröklés, polimorfizmus

Az Oracle ORDBMS támogatja az öröklést is. Egy altípust az UNDER kulcsszóval lehet létrehozni a szülőtípusból, feltéve, hogy a szülő NOT FINAL minősítéssel rendelkezik. Az altípus örökli a szülő összes attribútumát és metódusát, és kiegészítheti azokat saját tagokkal.

A polimorfizmus az OVERRIDING MEMBER FUNCTION segítségével valósul meg: az altípus felülírhatja a szülő metódusát. Így egy közös szülőtípust tartalmazó objektumtáblán ugyanaz a metódushívás minden sornál a tényleges (mögöttes) típusnak megfelelő kódot futtatja.

2.3. IS OF és TREAT operátorok

Polimorf objektumtáblák esetén az IS OF operátorral le lehet szűrni a sorokat egy adott típusra. Az IS OF (típus_t) TRUE értéket ad, ha a sor altípusa megfelel a megadott típusnak (az altípusokat is beleszámítva). Az ONLY kulcsszóval pontosan egy adott típusra szűrhetünk (altípusok nélkül).

A TREAT operátor típuskonverziót végez: a szülőtípusú értéket altípusként kezeli, így elérhetők az altípus saját attribútumai. Ha a konverzió nem sikerül, NULL értéket ad vissza (nem hibát). Ez biztonságos alternatíva a CAST operátorral szemben, amely hibát dob sikertelen konverzió esetén.

3. fejezet

Gyakorlati feladatok megoldása

3.1. Alaptípus létrehozása (szemely_t)

Az első feladatként egy `szemely_t` nevű ős objektumtípust kell létrehozni, amely NOT FINAL minősítéssel rendelkezik (ez kötelező az örökléshez). A típus egy nev attribútumot tartalmaz, és három metódust definiál a TYPE BODY-ban.

A `bemutakozas()` MEMBER FUNCTION visszatér a személy nevével, a `min_munkakor_eletkor()` STATIC FUNCTION a konstans minimális munkavállalási életkort (18) adja vissza példányosítás nélkül, az `osszehasonlit()` ORDER MEMBER FUNCTION pedig két `szemely_t` objektumot hasonlít össze név alapján, -1/0/1 értékkel.

Programkód 3.1. `szemely_t` alaptípus definiálása

```
CREATE OR REPLACE TYPE szemely_t AS OBJECT (  
    nev VARCHAR2(100),  
    MEMBER FUNCTION bemutakozas RETURN VARCHAR2,  
    STATIC FUNCTION min_munkakor_eletkor RETURN NUMBER,  
    ORDER MEMBER FUNCTION osszehasonlit(masik_szemely szemely_t) RETURN INTEGER  
) NOT FINAL;  
/
```

3.2. Öröklés: foszakacs_t és vendeg_t típusok

A második feladatban két altípust kell létrehozni `szemely_t` alaptípusból az UNDER kulcsszóval. A `foszakacs_t` típus `fkod` és `eletkor` attribútumokkal egészíti ki az örökölt `nev` mezőt, és felülírja a `bemutakozas()` függvényt úgy, hogy pozíciót és életkort is kiír.

A `vendeg_t` típus hasonlóan `vkod` és `eletkor` attribútumokat kap, `bemutakozas()` felülírásával pedig egyszerűen közli, hogy vendég. A polimorfizmus lényege jól megfigyelhető, hiszen mindkét altípusnál ugyanaz a metódusnév hívódik meg, de különböző szöveget ad vissza.

Programkód 3.2. Altípusok létrehozása

```
CREATE OR REPLACE TYPE foszakacs_t UNDER szemely_t (  
    fkod VARCHAR2(10),
```

```

    életkor NUMBER,
    OVERRIDING MEMBER FUNCTION bemutatkozas RETURN VARCHAR2
);
/

```

3.3. Objektumtábla létrehozása és feltöltése XML-ből

A harmadik feladatban létre kell hozni az `etterem_szemelyek_tbl` objektumtáblát a `szemely_t` közös ős típus alapján. Ez a tábla polimorf, képes tárolni a `szemely_t` összes altípusának példányaait egyazon táblában.

A feltöltés `XMLTABLE()` függvénnyel történik egy meglévő `vendeglatas_xml` táblából. Először a vendégeket szűrjük be a `vendeg_t` konstruktorával, majd a főszakácsokat a `foszakacs_t` konstruktorával. Az `XMLTABLE` az XML dokumentumból XPath kifejezésekkel emeli ki az egyes attribútumokat.

Programkód 3.3. Adatbetöltés polimorf táblába

```

CREATE TABLE etterem_szemelyek_tbl OF szemely_t;

INSERT INTO etterem_szemelyek_tbl
SELECT vendeg_t(x.nev, x.vkod, x.eletkor)
FROM vendeglatas_xml v,
     XMLTABLE('/vendeglatas/vendeg'
              PASSING v.adat
              COLUMNS
                 vkod      VARCHAR2(10)  PATH '@vkod',
                 nev       VARCHAR2(100)  PATH 'nev',
                 életkor   NUMBER         PATH 'eletkor'
              ) x;

```

3.4. Polimorf lekérdezések

A negyedik feladatban négy különböző lekérdezést kell készíteni. A statikus függvény a `DUAL` táblán keresztül hívható meg:

```

SELECT szemely_t.min_munkakor_eletkor() AS min_eletkor FROM DUAL;

```

A polimorf `bemutatkozas()` hívás és ABC sorrendű rendezés az `ORDER MEMBER FUNCTION`-t aktiválja, amit a `VALUE()` függvény és az `ORDER BY` kombináció tesz lehetővé:

```

SELECT s.nev, s.bemutatkozas() AS bemutatkozas
FROM etterem_szemelyek_tbl s
ORDER BY VALUE(s);

```

A vendégek szűréséhez az `IS OF` operátor alkalmazható, a `TREAT()`-tel pedig elérhető az életkor, amely az ős típusban nem szerepel:

```
SELECT s.nev
FROM etterem_szemelyek_tbl s
WHERE VALUE(s) IS OF (vendeg_t);

SELECT s.nev, TREAT(VALUE(s) AS vendeg_t).eletkor AS eletkor
FROM etterem_szemelyek_tbl s
WHERE VALUE(s) IS OF (vendeg_t);
```

3.5. szakacs2_t altípus integrálása

Az ötödik, önálló feladatban a sima szakácsokat kell elkészíteni az előzőekhez hasonlóan. Azért `szakacs2_t` a név, mert egy korábbi gyakorlaton már létrehoztunk egy `szakacs_t` típust, így az a név foglalt volt.

Az új altípus `szkod` (`VARCHAR2`) és `reszleg` (`VARCHAR2`) attribútumokat kap, és felülírja a `bemutakozas()` függvényt a `reszleg` megjelölésével. Az XML-ből való betöltés és a lekérdezés az előző feladatokhoz hasonlóan zajlik, az `IS OF` operátorban több típust is felsorolható vesszővel elválasztva:

Programkód 3.4. Szakácsok szűrése

```
SELECT s.nev, s.bemutakozas() AS bemutakozas
FROM etterem_szemelyek_tbl s
WHERE VALUE(s) IS OF (foszakacs_t, szakacs2_t);
```